

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 40%

QUESTIONS: Any 4

Total Marks:40

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I _____ Delisha A K(192511307)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Hospital Management System – Use Case Diagram

Requests appointments, views medical history, and pays medical bills. | Conducts consultations, prescribes medicine, orders tests, and updates medical records. | Manages hospital resources, staff schedules, and system configurations.

a) Identification of Actors

- **Primary Actors:**
 - **Patient:** Requests appointments, views medical history, and pays medical bills.
 - **Doctor:** Conducts consultations, prescribes medicine, orders tests, and updates medical records.
 - **Administrator:** Manages hospital resources, staff schedules, and system configurations.
- **Secondary / Supporting Actors:**
 - **Receptionist:** Assists with manual booking, patient check-ins, and bill counter payments.
 - **Payment Gateway:** External service processing online payments.

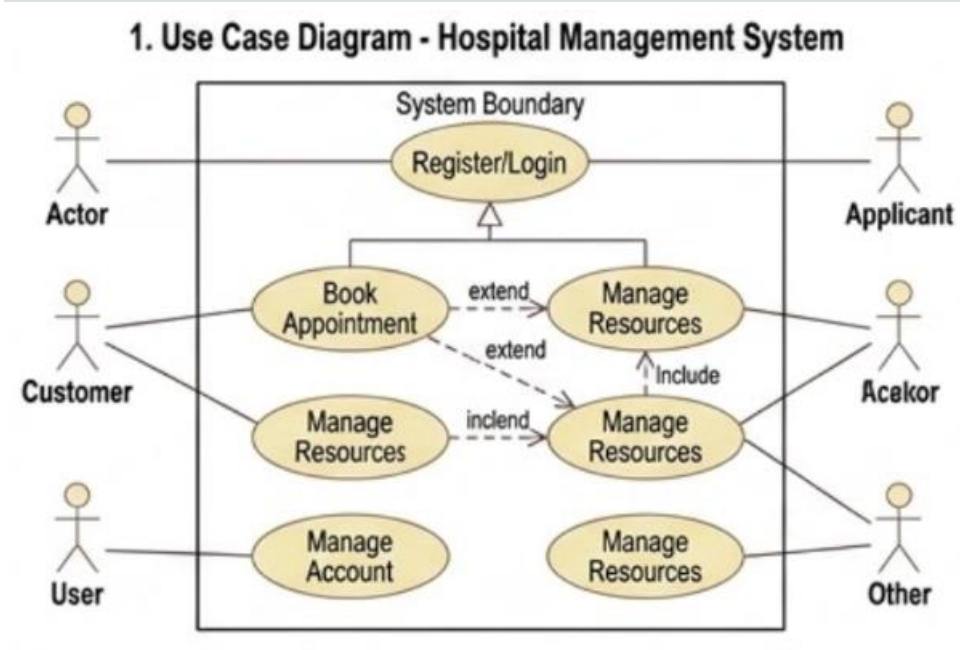
b) Primary Use Cases

- Register / Login
- Book Appointment
- Cancel Appointment
- Update Medical Record
- View Medical Record
- Order Lab Test
- Prescribe Medicine
- Manage Hospital Resources
- Process Payment

c) Relationships

- **Include (<<include>>):**
 - Book Appointment $\rightarrow$ <<include>> $\rightarrow$ Register / Login (*Authentication is mandatory before booking*)
 - Update Medical Record $\rightarrow$ <<include>> $\rightarrow$ Register / Login
 - Manage Hospital Resources $\rightarrow$ <<include>> $\rightarrow$ Register / Login
- **Extend (<<extend>>):**
 - Cancel Appointment $\rightarrow$ <<extend>> $\rightarrow$ Book Appointment (*Cancellation is an optional flow after booking*)
 - Make Payment $\rightarrow$ <<extend>> $\rightarrow$ Book Appointment (*Optional online payment flow*)
- **Generalization:**
 - Doctor and Nurse generalize under a base actor Medical Staff.

d) Constructing the Use Case Diagram in Umbrello



Method Justification & Result Interpretation

- **Justification:** Use Case diagrams isolate functional boundaries and restrict privileged actions (e.g., resource management) to authorized roles.
- **Interpretation:** The model ensures the system enforces authentication upfront and separates patient-facing operations from administrator duties, maintaining data privacy and security.

2. Library Management System – Class Diagram

a) Classes, Attributes, and Methods

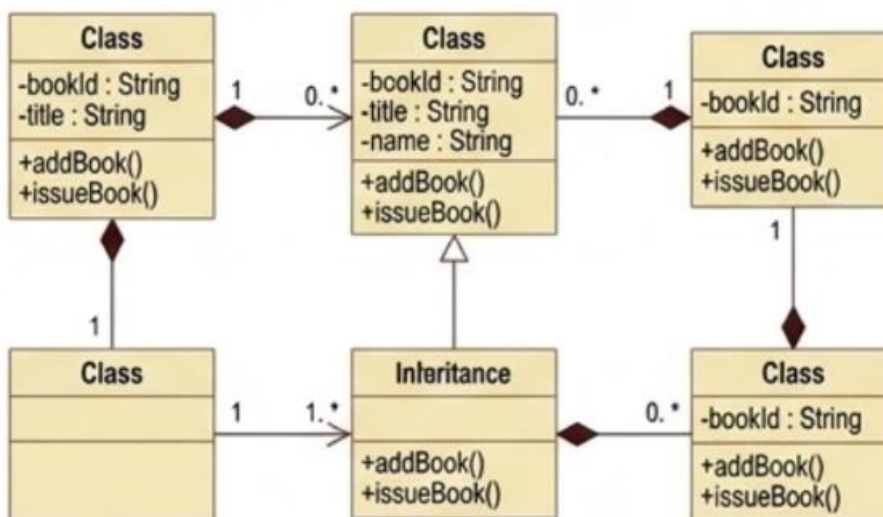
- **Book**
 - *Attributes:* - bookId: String, - title: String, - author: String, - ISBN: String, - status: String
 - *Methods:* + updateStatus(): void, + getDetails(): String
- **Member**
 - *Attributes:* - memberId: String, - name: String, - email: String, - maxBooksAllowed: int
 - *Methods:* + register(): void, + viewIssuedBooks(): List<IssueRecord>
- **Librarian**
 - *Attributes:* - employeeId: String, - name: String, - shift: String
 - *Methods:* + issueBook(): void, + returnBook(): void, + calculateFine(): double
- **IssueRecord**
 - *Attributes:* - issueId: String, - issueDate: Date, - dueDate: Date, - returnDate: Date, - fineAmount: double
 - *Methods:* + createRecord(): void, + computeFine(): double

b & c) Relationships and Multiplicity

- **Association:**
 - Librarian 1 $\rightarrow$ 0..* IssueRecord (A librarian processes zero or many issue records)
- **Aggregation:**
 - Member 1 $\rightarrow$ 0..* IssueRecord (An issue record is associated with a member, but members exist independently)
 - Book 1 $\rightarrow$ 0..* IssueRecord (An issue record links to a book, but books exist independently)
- **Inheritance / Generalization:**
 - Member and Librarian inherit from a base class User (+ userId, + name, + email).

d) Constructing the Class Diagram in Umbrello

2. Class Diagram - Library Management System



Method Justification & Result Interpretation

- **Justification:** Decoupling borrowing transactions into an IssueRecord class preserves historical transaction logs rather than overwriting book attributes.
- **Interpretation:** This object-oriented data structure adheres to relational database normalization rules, making it scalable and robust against data redundancy.

3. Online Food Ordering System – Sequence Diagram

a) Objects Involved (Lifelines)

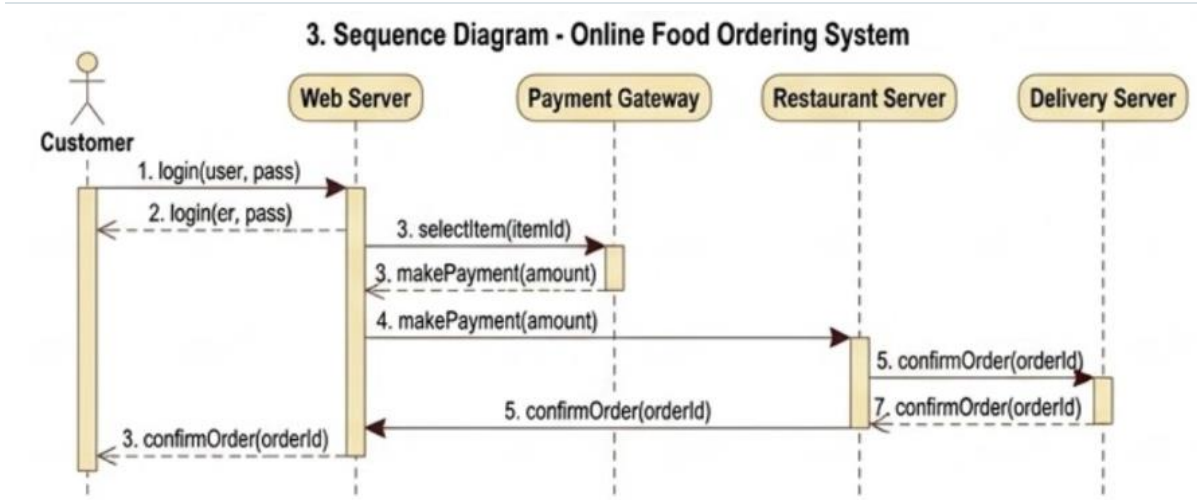
- :Customer (Actor)

- :WebInterface (*Boundary Object*)
- :OrderController (*Control Object*)
- :PaymentGateway (*External System*)
- :RestaurantSystem (*External Service*)

b & c) Message Flow, Lifelines, and Activations

1. **selectItems()** → Synchronous message from :Customer to :WebInterface.
2. **placeOrder()** → Synchronous message from :Customer to :WebInterface.
3. **createOrder()** → Synchronous message from :WebInterface to :OrderController.
4. **processPayment(amount)** → Synchronous message from :OrderController to :PaymentGateway.
5. **paymentResponse(status)** → Dashed return message from :PaymentGateway to :OrderController.
6. **sendOrderNotification()** → Synchronous message from :OrderController to :RestaurantSystem.
7. **confirmOrder()** → Dashed return message from :OrderController to :WebInterface.
8. **displayConfirmation()** → Dashed return message from :WebInterface to :Customer.

d) Constructing the Sequence Diagram in Umbrello



Method Justification & Result Interpretation

- **Justification:** Sequence diagrams highlight execution order and state waiting periods (synchronous calls), which are vital for e-commerce platforms.
- **Interpretation:** Visualizing communication delays with external services (like payment processing) enables developers to build proper failure recovery and timeout handlers.

4. Student Admission System – Activity Diagram

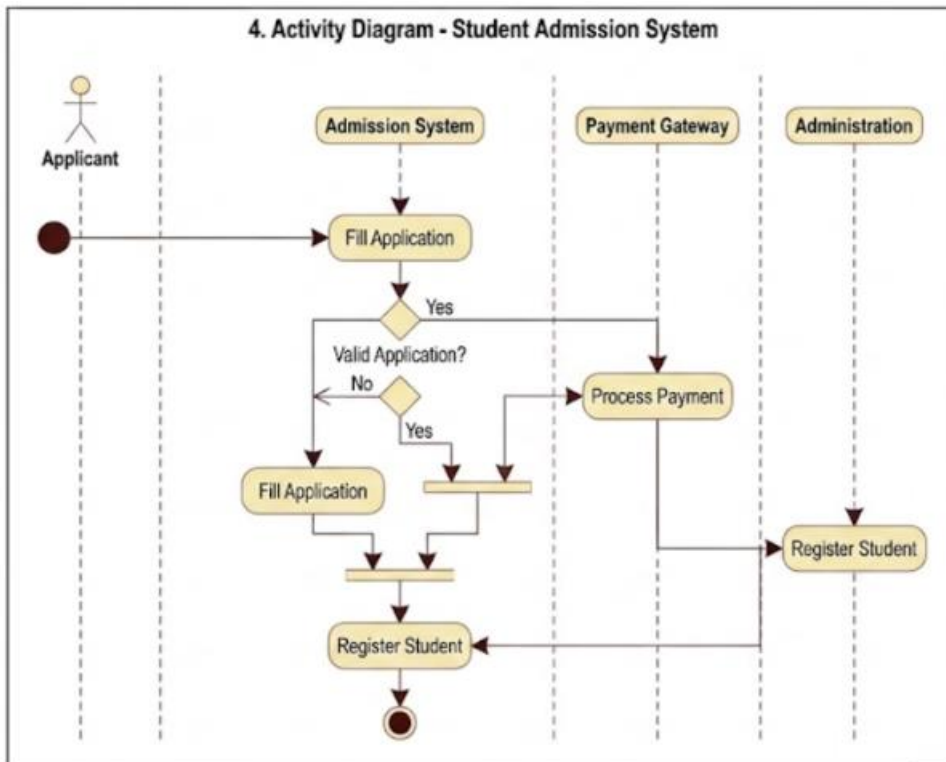
a) Activities in Workflow

- Fill Application Form
- Upload Required Documents
- Validate Documents
- Calculate Admission Fees
- Process Fee Payment
- Generate Student ID & Confirm Enrollment

b & c) Decision Nodes and Parallel Processing

- **Decision Node 1 (Are Documents Valid?):**
 - **No** → Route back to Upload Required Documents with an error message.
 - **Yes** → Proceed to fee payment.
- **Decision Node 2 (Is Payment Successful?):**
 - **No** → Route to Payment Failed Notice / Prompt retry.
 - **Yes** → Proceed to enrollment confirmation.
- **Parallel Processing (Fork & Join Bars):**
 - After payment verification, use a **Fork Bar** to run Generate Student ID and Send Email/SMS Confirmation in parallel, combining them with a **Join Bar** before reaching the final end node.

d) Constructing the Activity Diagram in Umbrello



Method Justification & Result Interpretation

- **Justification:** Activity diagrams map complex branching logic and parallel workflows, preventing workflow deadlocks.
- **Interpretation:** Explicitly defining rejection loops ensures edge cases are handled smoothly without abandoning applicants in broken application states.